Software Test Plan and Execution Record

Traffic Management System Simulation
Version 1.1 | 11 November 2026
Team: Aarya, Vijeth, Vaibhav and Vainik

1 Introduction

This plan covers all 18 FRs and 10 NFRs for the standalone simulation. It preserves the original 28 case IDs and procedures and adds current execution evidence. Original plan authors: Aarya and Vainik. Implementation evidence is recorded; team acceptance and unfamiliar-user usability review remain pending.

2 Test items

Scenario parsing, vehicles and queues; roads, routing and movement; signal timing and priority; incidents and congestion response; metrics and reports; deterministic reset; input robustness, Linux portability, traceability and build tooling.

3 Features to be tested

FR-01 through FR-18 and NFR-01 through NFR-10 are in scope. Include both isolated deterministic modules and integrated CLI scenarios. See Section 13 for original cases, source mapping and execution status.

4 Features not to be tested

Physical traffic hardware, live GPS/CCTV, machine-learning prediction, detailed physics and remote authentication are outside the project. GUI usability and first-class bidirectional road records are not acceptance requirements.

5 Test approach and strategy

Use GoogleTest unit tests, CTest integration/CLI regressions, deterministic system fixtures and code/document review. Test safe clearance and conflicting phases, full roads/lanes, no-route cases, invalid input, output failures, incident restoration, congestion responses and seeded reset. The benchmark uses four intersections and 500 generated vehicles. Each failure blocks acceptance until fixed or explicitly accepted by the team.

5.1 Security validation

SEC-01: malformed and out-of-range values, wrong references and signal movements must fail safely. SEC-02: file, line, duration and potential-arrival limits must be enforced. SEC-03: inspect RAII and run warnings, formatting, analysis and file open/write/close failure checks. SEC-04: priority must retain clearance, configured conflict safety and movement capacity. No TLS or banking tests apply to this local simulation.

6 Test environment

Linux-7.0.0-38-generic-x86_64-with-glibc2.43; c++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0. C++17 Debug and GCC/gcov coverage builds; GoogleTest v1.15.2. See reports/system-validation.json for source fingerprint and runtime output. Scripts require Python 3 and CMake/CTest 3.21 or newer for JUnit export.

7 Test schedule and execution

Original planning dates remain in repository history. This execution record was prepared on 11 November 2026 after feature integration. Unit, integration, CLI, system, coverage and static checks were run on this revision. External pipeline runs, final owner approval and unfamiliar-user review are pending; no completion date is invented for these activities.

8 Test deliverables

docs/reports/unit-tests.md, system-validation.md and JSON, coverage/code-coverage.md, coverage/branch-coverage.md, machine-readable JUnit/coverage data, benchmark/acceptance CSVs, traceability.md and submission-checklist.md. Demo videos are current capability recordings rather than claimed historical sprint evidence.

9 Roles and responsibilities

Owner	Responsibility
Aarya	Input/vehicle requirements, CMake/coverage, Test Plan/RTM with Vainik.
Vijeth	Signal safety/priority, static checks and repository-owner review.
Vaibhav	Network/movement/routing, reproducible container.
Vainik	Incidents/congestion/metrics/reports, Jenkins/archive, Test Plan with Aarya.
All	System validation, board/sprint evidence, demo/viva and final acceptance.

10 Risks and mitigation

Runtime/tool availability may block container or Jenkins validation; keep those states separate from native test results. Performance is workload-specific. Continuous emergency demand can delay normal phases. Coverage includes compiler exception paths, so branch coverage is lower than line coverage and must be reported without exclusions.

11 Assumptions and dependencies

Use the documented text-file CLI and directed network. Signal plans must define all needed movements and the full conflict table. First testing configure requires GoogleTest download or --gtest-source. External runners need GCC/gcov, Clang, Python, pinned clang-format and a running Docker engine.

12 Suspension and resumption

Suspend acceptance when builds/tests fail, invalid state is observed or required evidence is unavailable. Resume after a fix, relevant regression pass and evidence review. Passing native tests does not waive external or human acceptance checks.

13 Traceability and original case execution

Exact source and executable test mappings are maintained in traceability.md. The original cases below retain their preconditions, steps and expected behavior; execution results refer to the recorded automated suite and system checks. Human usability review is explicitly pending.

TC-01 | FR-01 | Load valid and invalid scenarios

Preconditions / data: Use one valid scenario and fixtures with a missing road endpoint, negative capacity, and invalid duration.

Steps: Load each fixture through the supported scenario interface.

Expected: The valid scenario loads; each invalid configuration is rejected with a clear error and no crash.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-02 | FR-02 | Generate vehicles to a configured schedule

Preconditions / data: Load a small scenario with a deterministic vehicle schedule and known arrival ticks.

Steps: Start the simulation and inspect generated vehicles at each scheduled tick.

Expected: Vehicles are generated according to the configured schedule; no unscheduled generation occurs in this deterministic case.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-03 | FR-03 | Assign unique and valid vehicle attributes

Preconditions / data: Configure multiple vehicles with valid sources, destinations, classes, and connected routes.

Steps: Generate the vehicles and inspect their identifiers and required attributes.

Expected: Every generated vehicle has a unique ID and the configured valid source, destination, class, and route data.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-04 | FR-04 | Enforce entry-lane capacity

Preconditions / data: Set an entry lane capacity to 2 and schedule 3 vehicles for that lane.

Steps: Advance generation while inspecting the lane queue and handling of the third vehicle.

Expected: The queue never exceeds 2; the additional vehicle is not inserted beyond capacity and is handled safely.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-05 | FR-05 | Maintain valid signal state per intersection

Preconditions / data: Load a scenario with controlled intersections and configured signal phases.

Steps: Initialize the simulation and inspect each intersection signal state.

Expected: Each controlled intersection has a valid phase/state defined by its configuration.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-06 | FR-06 | Advance signal timers without conflicting greens

Preconditions / data: Configure an intersection with conflicting movements and valid phase timings.

Steps: Advance ticks through multiple transitions and inspect active phases and timers at each step.

Expected: Transitions follow configured timing rules and conflicting green phases are never active simultaneously.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-07 | FR-07 | Adapt green timing within configured bounds

Preconditions / data: Prepare two otherwise comparable cases with different queue lengths and configured minimum/maximum green durations.

Steps: Run adaptive control and record the chosen durations and queue inputs.

Expected: Queue information affects the extension/shortening decision, and every selected duration remains within the configured minimum and maximum.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-08 | FR-08 | Grant emergency priority safely

Preconditions / data: Create a valid emergency-priority request for a movement that conflicts with another approach.

Steps: Submit the request while the conflicting approach is active; observe signal handling.

Expected: The request triggers the configured priority behavior, while no conflicting green phase is enabled.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-09 | FR-09 | Represent connected road network data

Preconditions / data: Load a small graph with known nodes, edges, capacities, and travel times.

Steps: Query or inspect the network through the road-network interface.

Expected: Configured connections, capacities, and travel times are accessible and match the scenario data.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-10 | FR-10 | Move eligible vehicles along planned routes

Preconditions / data: Place a vehicle at a connected approach with a valid route, green signal, and downstream space.

Steps: Advance the simulation through the movement opportunity.

Expected: The vehicle advances along its planned connected route only when signal and capacity conditions permit.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-11 | FR-11 | Block movement into a full downstream lane

Preconditions / data: Fill the downstream lane to its configured capacity and place an eligible vehicle upstream.

Steps: Advance the simulation while inspecting both queue occupancies.

Expected: The upstream vehicle does not enter the full downstream lane; occupancy never exceeds configured capacity.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-12 | FR-12 | Select a minimum-cost connected route

Preconditions / data: Create a graph with at least two connected routes whose configured route costs are known and unequal.

Steps: Request a route to the destination using the configured shortest/weighted-path policy.

Expected: The selected route consists of connected edges and has the minimum configured route cost.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-13 | FR-13 | Detect congestion at configured threshold

Preconditions / data: Configure a queue/occupancy threshold and prepare observations immediately below and at/above it.

Steps: Evaluate congestion for each observation.

Expected: Conditions below the configured threshold are not flagged; conditions meeting the threshold are flagged.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-14 | FR-14 | Mark blocked road unavailable

Preconditions / data: Load a scenario containing a road segment used by a planned route.

Steps: Inject a road blockage and inspect road state and attempted movements through that segment.

Expected: The affected segment becomes unavailable or restricted and prohibited movement through it is prevented.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-15 | FR-15 | Respond to incident with rerouting and no-route handling

Preconditions / data: Prepare one network with an alternate route and another where blockage removes all routes to the destination.

Steps: Trigger an incident or congestion response in both networks.

Expected: The configured response activates; the first vehicle is rerouted as supported, and the no-route case is handled safely without invalid movement or a crash.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-16 | FR-16 | Collect traffic and trip metrics

Preconditions / data: Run a small deterministic scenario with known vehicle completions, waits, and queue observations.

Steps: Compare reported throughput, waiting time, queue statistics, and completed/unfinished trip counts against recorded events.

Expected: Each metric reflects the simulation events and final vehicle states for the scenario.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-17 | FR-17 | Produce readable text or CSV summary

Preconditions / data: Complete a scenario with known metrics and select each supported output format.

Steps: Generate and open the final text and/or CSV report.

Expected: The report is readable and contains the required metrics; CSV records use consistent labels and valid row/column structure.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-18 | FR-18 | Repeat equivalent simulation with the same seed

Preconditions / data: Use a fixed scenario configuration and deterministic seed.

Steps: Run the same simulation twice and compare generated traffic and final results.

Expected: Both runs produce equivalent results under deterministic execution conditions.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-01 | NFR-01 | Verify clear module interfaces and controlled shared state

Preconditions / data: Review documented module boundaries and the integrated source when available.

Steps: Inspect interfaces and state ownership across modules, including mutable data shared between modules.

Expected: Module communication uses clear defined interfaces; uncontrolled global mutable state is absent outside the controlled simulation context.

Actual: Public interfaces, controlled state and exact source/test mapping reviewed against the implementation. See architecture.md and traceability.md. Status: Reviewed; formal team sign-off pending.

TC-NFR-02 | NFR-02 | Test each module independently and deterministically

Preconditions / data: Select deterministic public methods in each available module and prepare isolated fixtures.

Steps: Run module tests without starting the entire simulation; repeat equivalent input/output cases.

Expected: Each module can be tested independently and produces the expected deterministic outputs.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-03 | NFR-03 | Run the standard performance scenario

Preconditions / data: Use a normal student laptop and a configured scenario with at least 4 intersections and 500 vehicles.

Steps: Run the scenario to completion; record hardware, build, seed, elapsed time, and any stalls.

Expected: The scenario completes without noticeable hangs; elapsed time and environment details are recorded for evaluation. The SRS defines no numeric time limit.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-04 | NFR-04 | Reject invalid configuration safely

Preconditions / data: Prepare invalid capacities, impossible timings, and references to unknown roads/intersections.

Steps: Load or submit each invalid value through the supported interface.

Expected: Every invalid configuration is rejected with a clear error and without a crash or undefined behavior.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-05 | NFR-05 | Build with C++17 and CMake on Linux

Preconditions / data: Use a clean Linux build directory and the project-supported C++17 compiler.

Steps: Configure with CMake and build the project using the documented commands.

Expected: The project configures and builds successfully as C++17 on Linux without relying on compiler-specific extensions.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-06 | NFR-06 | Review console and report usability

Preconditions / data: Use the console menu/commands and inspect a representative simulation report.

Steps: Ask a reviewer unfamiliar with the run to locate key commands, understand an error, identify simulation status, and interpret metric labels.

Expected: Commands, errors, status, and report labels are readable, consistent, and interpretable.

Actual: CLI help, errors, trace and report checks pass. Unfamiliar-user task review has not been performed. Status: Pending human review.

TC-NFR-07 | NFR-07 | Validate input handling and resource safety

Preconditions / data: Prepare malformed and boundary menu/file input, impossible values, and file-operation success/failure paths.

Steps: Exercise validation paths; inspect critical input code for unsafe C-style string use and resource management.

Expected: Invalid inputs are rejected safely; unsafe C-style string handling is absent; acquired resources are managed with RAII and released on success/failure.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-08 | NFR-08 | Check requirement-to-source-to-test traceability

Preconditions / data: Use the implemented requirement list, source tree, and Section 13 RTM after implementation.

Steps: For each implemented FR, verify the requirement ID maps to source file(s) and test ID(s); note any unimplemented requirement explicitly.

Expected: Every implemented FR has a source-file reference and test ID in the RTM. Source references remain pending until implementation files exist.

Actual: Public interfaces, controlled state and exact source/test mapping reviewed against the implementation. See architecture.md and traceability.md. Status: Reviewed; formal team sign-off pending.

TC-NFR-09 | NFR-09 | Review warnings, formatting, and static analysis

Preconditions / data: Use the project build and the tools/commands selected by the development team.

Steps: Build with configured warnings; check formatting consistency; run static analysis and triage findings.

Expected: Warnings are enabled, formatting is consistent, and no critical static-analysis finding remains unresolved.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

TC-NFR-10 | NFR-10 | Record and reuse the random seed

Preconditions / data: Run a scenario that uses random vehicle generation with a specified seed.

Steps: Check the displayed/stored seed, then repeat with the same configuration and seed and compare generated traffic/results.

Expected: The seed used is printed or stored, and repeated runs with the same seed and configuration produce equivalent traffic and results.

Actual: Corresponding automated/review evidence passed; exact test IDs and outputs are in traceability.md and docs/reports. Status: Pass for recorded implementation scope.

14 Test metrics and reporting

The recorded suite passed 115/115 CTest cases, including unit, integration and CLI cases. The acceptance runner passed 12/12 system checks. Coverage is measured separately and includes compiler exception branches; use the adjacent generated reports for actual percentages rather than a target presented as a result. All 28 original cases have a recorded outcome; human usability and formal approval remain open.

15 Approvals

Role	Name	Approval
Test Plan owners	Aarya / Vainik	Pending review
Repository owner	Vijeth	Pending review
Course evaluator	As assigned by course	Pending evaluation